Overture Maps Data Download

Applying DuckDB in R and Python

Pukar Bhandari

pukar.bhandari@outlook.com

2024-05-22

Table of contents

Downloading Overture Maps Data with R and Python 1
- What is Overture Maps? 1
- Prerequisites 2
- Setting Up the Environment 2
- Data Type Mapping 2
- Core Download Function 3
- Defining Your Study Area 6
- Downloading the Data 7
- Processing Downloaded Data 7
- Data Visualization 8
- Available Data Types 9
- Transportation Planning Applications 9
- Repository and Additional Resources 9

Downloading Overture Maps Data with R and Python

Overture Maps Foundation provides a collaborative, open-source initiative to create the world's most comprehensive and interoperable geospatial dataset. As transportation planners and data analysts, we often need access to high-quality geospatial data for buildings, transportation networks, places, and administrative boundaries. This post demonstrates how to efficiently download Overture Maps data using both R and Python with DuckDB's powerful spatial capabilities.

What is Overture Maps?

Overture Maps is an open-source mapping initiative that provides global-scale geospatial data across four main themes:

- **Buildings**: Footprints and building parts
- **Transportation**: Road segments and connectors
- **Places**: Points of interest and place data
- **Admins**: Administrative boundaries and localities

- **Base**: Infrastructure, land use, land cover, and water features

The data is stored in cloud-optimized Parquet format on AWS S3, making it ideal for efficient querying and analysis.

Prerequisites

Before diving into the code, ensure you have the following dependencies installed:

R

```
# Install required packages
install.packages(c("tidyverse", "sf", "mapview", "DBI", "duckdb", "arrow"))
```

Python

```
# Install required packages
pip install duckdb matplotlib geopandas pandas shapely folium pathlib fastparquet
```

Setting Up the Environment

First, we need to load our libraries and configure the environment for spatial data processing.

R

```
# Load required libraries
library(tidyverse)
library(sf)
library(mapview)
library(DBI)
library(duckdb)
library(arrow)
```

Python

```
import duckdb
import geopandas as gpd
import pandas as pd
import shapely.wkb
import matplotlib.pyplot as plt
import folium
import os
from pathlib import Path
```

Data Type Mapping

Overture data is organized by themes, and we need to map specific data types to their corresponding themes for proper S3 path construction.

R

```r
# Define the theme map
map_themes <- list(
  "locality" = "admins",
  "locality_area" = "admins",
  "administrative_boundary" = "admins",
  "building" = "buildings",
  "building_part" = "buildings",
  "place" = "places",
  "segment" = "transportation",
  "connector" = "transportation",
  "infrastructure" = "base",
  "land" = "base",
  "land_use" = "base",
  "water" = "base"
)
```

Python

```python
# Define theme mapping
map_themes = {
    "locality": "admins",
    "locality_area": "admins",
    "administrative_boundary": "admins",
    "building": "buildings",
    "building_part": "buildings",
    "place": "places",
    "segment": "transportation",
    "connector": "transportation",
    "infrastructure": "base",
    "land": "base",
    "land_use": "base",
    "water": "base",
}
```

Core Download Function

This function handles the DuckDB connection, S3 configuration, and spatial filtering to download only the data within your specified bounding box.

R

```r
overture_data <- function(bbox, overture_type, dst_parquet) {
```

```
  # Validate overture_type
  if (!overture_type %in% names(map_themes)) {
    stop(paste("Valid Overture types are:", paste(names(map_themes), collapse =
", ")))
  }

  # Configure S3 path
  s3_region <- "us-west-2"
  base_url <- sprintf("s3://overturemaps-%s/release", s3_region)
  version <- "2024-04-16-beta.0"
  theme <- map_themes[[overture_type]]
  remote_path <- sprintf("%s/%s/theme=%s/type=%s/*", base_url, version, theme,
overture_type)

  # Connect to DuckDB and install extensions
  conn <- DBI::dbConnect(duckdb::duckdb())
  DBI::dbExecute(conn, "INSTALL httpfs;")
  DBI::dbExecute(conn, "INSTALL spatial;")
  DBI::dbExecute(conn, "LOAD httpfs;")
  DBI::dbExecute(conn, "LOAD spatial;")
  DBI::dbExecute(conn, sprintf("SET s3_region='%s';", s3_region))

  # Create view and execute spatial query
          read_parquet     <-     sprintf("read_parquet('%s',     filename=TRUE,
hive_partitioning=1);", remote_path)
   DBI::dbExecute(conn, sprintf("CREATE OR REPLACE VIEW data_view AS SELECT *
FROM %s", read_parquet))

  query <- sprintf("
    SELECT data.*
    FROM data_view AS data
    WHERE data.bbox.xmin <= %f AND data.bbox.xmax >= %f
    AND data.bbox.ymin <= %f AND data.bbox.ymax >= %f
  ", bbox[3], bbox[1], bbox[4], bbox[2])

  # Save results to Parquet file
  file <- normalizePath(dst_parquet, mustWork = FALSE)
   DBI::dbExecute(conn, sprintf("COPY (%s) TO '%s' WITH (FORMAT 'parquet');",
query, file))
  DBI::dbDisconnect(conn, shutdown = TRUE)
}
```

Python

```python
def overture_data(bbox, overture_type, dst_parquet):
    """Query a subset of Overture's data and save it as a GeoParquet file.

    Parameters
    ----------
    bbox : tuple
       A tuple of floats representing the bounding box (xmin, ymin, xmax, ymax)
        in EPSG:4326 coordinate reference system.
    overture_type : str
        The type of Overture data to query
    dst_parquet : str or Path
        The path to the output GeoParquet file.
    """
    if overture_type not in map_themes:
        raise ValueError(f"Valid Overture types are: {list(map_themes)}")

    # Configure S3 connection
    s3_region = "us-west-2"
    base_url = f"s3://overturemaps-{s3_region}/release"
    version = "2024-04-16-beta.0"
    theme = map_themes[overture_type]
    remote_path = f"{base_url}/{version}/theme={theme}/type={overture_type}/*"

    # Setup DuckDB with spatial extensions
    conn = duckdb.connect()
    conn.execute("INSTALL httpfs;")
    conn.execute("INSTALL spatial;")
    conn.execute("LOAD httpfs;")
    conn.execute("LOAD spatial;")
    conn.execute(f"SET s3_region='{s3_region}';")

    # Execute spatial query
           read_parquet   =   f"read_parquet('{remote_path}',   filename=true,
hive_partitioning=1);"
       conn.execute(f"CREATE  OR  REPLACE  VIEW  data_view  AS  SELECT  *  FROM
{read_parquet}")

    query = f"""
    SELECT data.*
    FROM data_view AS data
    WHERE data.bbox.xmin <= {bbox[2]} AND data.bbox.xmax >= {bbox[0]}
    AND data.bbox.ymin <= {bbox[3]} AND data.bbox.ymax >= {bbox[1]}
```

```
    """

    file = str(Path(dst_parquet).resolve())
    conn.execute(f"COPY ({query}) TO '{file}' WITH (FORMAT PARQUET);")
    conn.close()
```

Defining Your Study Area

For spatial analysis, you need to define a bounding box for your area of interest. This can come from existing boundary data or manual coordinates.

R

```
# Read existing boundary data (example: Salt Lake County, UT)
slco_boundary <- sf::read_sf(
  "data/SaltLakeCounty_Boundary.geojson"
) |>
  sf::st_transform("EPSG:4326")

# Extract bounding box coordinates (xmin, ymin, xmax, ymax)
slco_bbox <- slco_boundary |>
  sf::st_bbox() |>
  as.vector()

print(slco_bbox)
```

```
[1] -112.26017   40.41417 -111.55320   40.92187
```

Python

```
# Read existing boundary data (example: Salt Lake County, UT)
slco_boundary = gpd.read_file("data/SaltLakeCounty_Boundary.geojson") \
    .to_crs("EPSG:4326")

# Extract bounding box coordinates (xmin, ymin, xmax, ymax)
slco_bbox = slco_boundary.total_bounds

print(slco_bbox)
```

```
[-112.26016909   40.41417467 -111.55319996   40.92186613]
```

Downloading the Data

Now we can download specific data types for our study area. The function handles all the cloud connectivity and spatial filtering automatically.

R

```r
# Download places data for Salt Lake County
if (!file.exists("output/slco_places_r.parquet")) {
  overture_data(slco_bbox, "place", "output/slco_places_r.parquet")
}
```

Python

```python
# Download places data for Salt Lake County
if not os.path.exists("output/slco_places_py.parquet"):
    overture_data(slco_bbox, "place", "output/slco_places_py.parquet")
```

Processing Downloaded Data

After downloading, convert the Parquet files to spatial data formats for analysis and visualization.

R

```r
# Read the downloaded Parquet file
slco_places <- arrow::read_parquet("output/slco_places_r.parquet")

# Convert to sf object for spatial operations
slco_places_sf <- sf::st_as_sf(
  slco_places |> dplyr::select(-sources),
  geometry = slco_places$geometry,
  crs = "EPSG:4326"
)

# Basic data exploration
print(paste("Downloaded", nrow(slco_places_sf), "places"))
```

```
[1] "Downloaded 44355 places"
```

```r
print(colnames(slco_places_sf))
```

```
 [1] "id"          "geometry"    "bbox"        "version"     "update_time"
 [6] "names"       "categories"  "confidence"  "websites"    "socials"
[11] "emails"      "phones"      "brand"       "addresses"   "filename"
[16] "theme"       "type"
```

Python

```python
# Read the downloaded data
slco_places = pd.read_parquet("output/slco_places_py.parquet")

# Convert to GeoDataFrame
slco_places_gdf = gpd.GeoDataFrame(
    slco_places.drop(columns="geometry"),
    geometry=shapely.wkb.loads(slco_places["geometry"]),
    crs=4326,
)

# Basic exploration
print(f"Downloaded {len(slco_places_gdf)} places")
```

```
Downloaded 44355 places
```

```python
print(slco_places_gdf.columns.tolist())
```

```
['id', 'bbox', 'version', 'update_time', 'sources', 'names', 'categories',
'confidence', 'websites', 'socials', 'emails', 'phones', 'brand', 'addresses',
'filename', 'theme', 'type', 'geometry']
```

Data Visualization

Create quick visualizations to explore your downloaded data and verify the results.

R

```r
# Simple quick visualization using mapview
set.seed(123)

slco_places_sf |>
  dplyr::sample_n(1000) |>
  tidyr::unnest(cols = c("names", "categories")) |>
  dplyr::select(
    names_primary = "primary",
    categories_main = "main",
    confidence
  ) |>
  mapview::mapview(map.types = "CartoDB.Voyager", legend = FALSE)
```

Python

```
# Simple quick visualization using GeoPandas
(
    slco_places_gdf
    .sample(n=1000, random_state=123)
    .assign(  # names and categories are struct columns, read back as dicts
        names_primary=lambda d: d["names"].str["primary"],
        categories_main=lambda d: d["categories"].str["main"],
    )
    .loc[:, ["names_primary", "categories_main", "confidence", "geometry"]]
    .explore(tiles="CartoDB.Voyager", zoom_start=9)
)
```

Available Data Types

Overture Maps provides the following data types organized by theme:

Theme	Data Types	Description
Admins	`locality`, `locality_area`, `administrative_boundary`	Administrative boundaries and place hierarchies
Buildings	`building`, `building_part`	Building footprints and structural components
Places	`place`	Points of interest, businesses, and landmarks
Transportation	`segment`, `connector`	Road networks and transportation infrastructure
Base	`infrastructure`, `land`, `land_use`, `water`	Base map features and land cover

Transportation Planning Applications

This approach is particularly valuable for transportation planning workflows where you need to integrate multiple data sources for comprehensive analysis. The standardized schema and efficient spatial querying make it ideal for network analysis, land use integration, and multi-modal planning across different jurisdictions and scales.

Repository and Additional Resources

The complete code and examples are available in the Overture Data Download repository on GitHub.

For more information about Overture Maps:

- Official Documentation
- Data Schema Reference

- Community Forum

Want to contribute or suggest improvements? Visit the project repository at: https://github.com/arpuuk/overture-data-download